Project - Submission Sheet

Software Testing - Random Testing

Names: Harine Balamuralikrishnan
Group: 04

Software Testing - Summer 2024 - June 14, 2025 - Sibylle Schupp / Daniel Rashedi

Task 1 - Random Testing

a) Add the 4 generated Python tests of each group member to your repository in the corresponding folder. Add a README.md file that explains how to build / run the tests.

b) Provide screenshots of your terminals that indicate your execution of the new tests:

Answer:

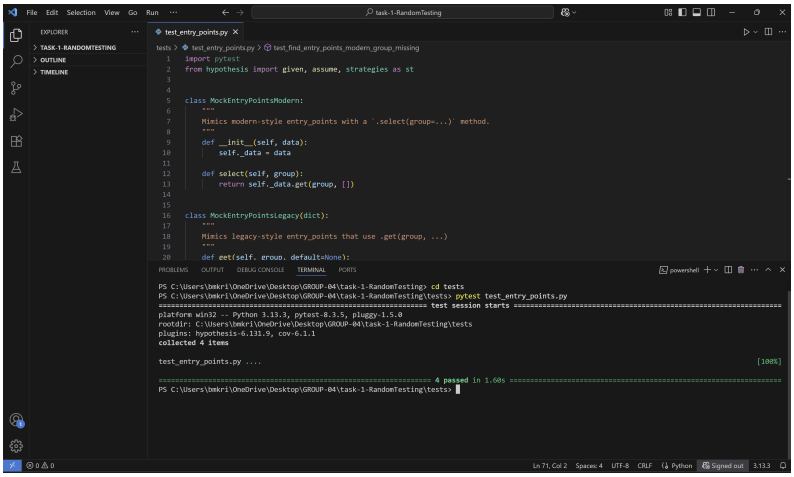

Figure 1: test_entry_points

c) Assess the input values generated by `hypothesis`. Do the tests represent meaningful scenarios?:

Answer:

Definitions

Input Strategy:
An input strategy defines how test data is generated automatically by Hypothesis. It specifies the range, type, and structure of inputs such as strings, lists, numbers, etc. For example, `st.text(min_size=1, max_size=5)` generates short strings of 1 to 5 characters.

Realistic Input:
Realistic inputs are values that reflect how a system is typically used in practice. These inputs simulate common user behaviors or real-world configurations. For example, using a short string like `"build"` as a group name is realistic for a plugin system.

Edge Case:
Edge cases refer to values at the boundaries or extremes of valid input. These are used to test how the software behaves under unusual or challenging conditions. Examples include empty strings, missing dictionary keys, or lists with no elements. Testing edge cases helps ensure robustness and error handling.

Answer:

TEST CASE	INPUT STRATEGY	REALISTIC	EDGE CASES	CONCLUSION
test_find_entry_points_modern	Group name (1–5 characters) and a list of strings (1–3 items) using `st.text`, `st.lists`	Yes	Yes (e.g., empty list, special characters)	Confirms correct behavior of modern entry point API via `.select(group)` and verifies the returned list matches expectations.
test_find_entry_points_legacy	Same as above, tested using legacy `.get(group)` API on a dictionary subclass	Yes	Yes (e.g., unordered list, duplicates)	Checks proper handling of legacy entry points and ensures the result is a `set` of expected values.
test_find_entry_points_modern_group_missing	Two random group names; `assume(group ≠ other_group)` ensures they don't match	Yes	Yes (group not found)	Tests if modern entry point API returns an empty list when the requested group is missing. Validates fail-safe behavior.
test_find_entry_points_legacy_group_missing	Similar strategy using legacy-style dictionary API with mismatched groups	Yes	Yes (missing key)	Ensures empty `set` is returned from legacy interface when the queried group is not found.

Figure 2: Execution result of tests in `test_entry_points.py`